Звіт

до лабораторної роботи 1

з навчальної дисципліни «Розподілені системи обробки інформації»

студента групи ММШІ-1 Красікова Антона

1. Постановка задачі

Реалізувати алгоритм HS-алгоритм пошуку лідера у синхронному кільці.

Вхідні дані: кількість процесів у кільці (n). Вихідні дані: UID процесу-переможця, кількість проведених раундів та сумарна кількість відправлених повідомлень по усім процесам. Додаткова вимога: дослідити залежність кількості повідомлень від кількості процесів та порівняти її з теоретичною складністю алгоритму.

Спосіб розв'язання:

Для розв'язання задачі використано HS-алгоритм, який дозволяє знизити комунікаційну складність з $O(n^2)$ (як у базовому алгоритмі LCR) до $O(nlog\ n)$. Алгоритм працює за наступним принципом:

1. Кожен процес має унікальний ідентифікатор (UID) і намагається стати лідером, обирається процес з максимальним UID.
2. Виконання розбито на фази *l = 0, 1, 2, ...* На кожній фазі процес відправляє токени зі своїм UID в обох напрямках по кільцю.
3. Токен має пройти відстань *2^l* кроків. Якщо на своєму шляху вихідний токен зустрічає вузол з більшим UID, токен знищується. Якщо зустрічає вузол з меншим UID, він продовжує рух.
4. Якщо токен успішно проходить *2^l* кроків, він розвертається і повертається до свого творця. Якщо процес отримує підтвердження з обох боків, він переходить до наступної фази.
5. Якщо процес отримує токен із власним UID, він проголошує себе лідером.

2. Деталі реалізації

Систему реалізовано мовою Python з використанням об'єктно-орієнтованого підходу для чіткого розділення відповідальності:

- **Клас `Message`**: Реалізований як immutable dataclass. Він переносить інформацію про UID, напрямок руху (INBOUND/OUTBOUND) та лічильник кроків, що залишились.
- **Клас `Node`**: Інкапсулює бізнес-логіку алгоритму HS. Вузол не має інформації про загальну топологію мережі, а спілкується зі світом виключно через абстрактні порти (LEFT, RIGHT) та власні буфери вхідних/вихідних повідомлень.
- **Клас `RingSimulator`**: Виконує роль середовища виконання. Він генерує унікальні UID, зв'язує вузли у кільце та керує глобальним часом (раундами). На кожному раунді симулятор синхронно переміщує повідомлення з вихідних черг до вхідних черг сусідів, гарантуючи детермінованість симуляції.

3. Результати

Алгоритм успішно знаходить вузол з максимальним UID у кільці будь-якого розміру.

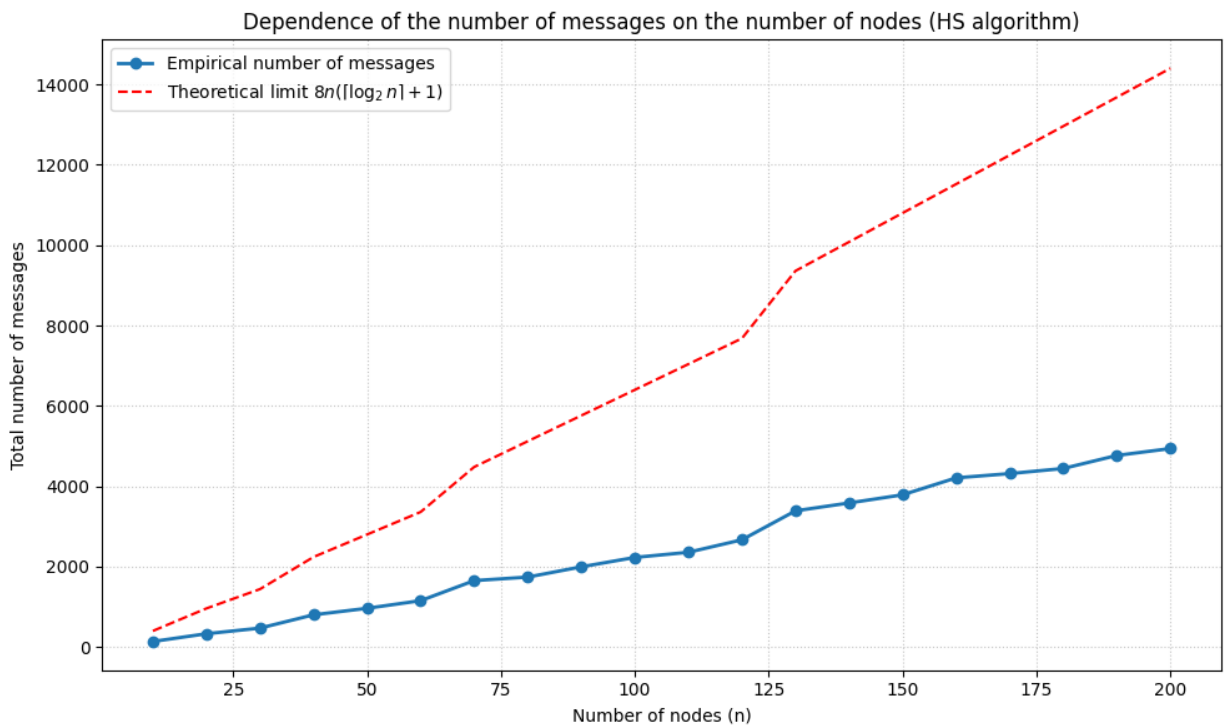
Приклад покрокового виконання для кільця з 5 процесів:

```
Start of the simulation. Node ids: [37, 29, 12, 25, 21].
[Round 0] Node 37 starts phase 0. Sending token to left and right for 1 steps.
[Round 0] Node 29 starts phase 0. Sending token to left and right for 1 steps.
[Round 0] Node 12 starts phase 0. Sending token to left and right for 1 steps.
[Round 0] Node 25 starts phase 0. Sending token to left and right for 1 steps.
[Round 0] Node 21 starts phase 0. Sending token to left and right for 1 steps.
[Round 1] Node 37 eliminated token 21.
[Round 1] Node 37 eliminated token 29.
[Round 1] Node 29 returns token 37 back (INBOUND).
[Round 1] Node 29 eliminated token 12.
[Round 1] Node 12 returns token 29 back (INBOUND).
[Round 1] Node 12 returns token 25 back (INBOUND).
[Round 1] Node 25 eliminated token 12.
[Round 1] Node 25 eliminated token 21.
[Round 1] Node 21 returns token 25 back (INBOUND).
[Round 1] Node 21 returns token 37 back (INBOUND).
[Round 2] Node 37 received an approval (1/2).
[Round 2] Node 37 received an approval (2/2).
[Round 2] Node 37 starts phase 1. Sending token to left and right for 2 steps.
[Round 2] Node 29 received an approval (1/2).
[Round 2] Node 25 received an approval (1/2).
[Round 2] Node 25 received an approval (2/2).
[Round 2] Node 25 starts phase 1. Sending token to left and right for 2 steps.
[Round 3] Node 29 sends token 37 forward (hop=1).
[Round 3] Node 12 sends token 25 forward (hop=1).
[Round 3] Node 21 sends token 25 forward (hop=1).
```

```
[Round 3] Node 21 sends token 37 forward (hop=1).
[Round 4] Node 37 eliminated token 25.
[Round 4] Node 29 eliminated token 25.
[Round 4] Node 12 returns token 37 back (INBOUND).
[Round 4] Node 25 returns token 37 back (INBOUND).
[Round 6] Node 37 received an approval (1/2).
[Round 6] Node 37 received an approval (2/2).
[Round 6] Node 37 starts phase 2. Sending token to left and right for 4 steps.
[Round 7] Node 29 sends token 37 forward (hop=3).
[Round 7] Node 21 sends token 37 forward (hop=3).
[Round 8] Node 12 sends token 37 forward (hop=2).
[Round 8] Node 25 sends token 37 forward (hop=2).
[Round 9] Node 12 sends token 37 forward (hop=1).
[Round 9] Node 25 sends token 37 forward (hop=1).
[Round 10] Node 29 returns token 37 back (INBOUND).
[Round 10] Node 21 returns token 37 back (INBOUND).
[Round 14] Node 37 received an approval (1/2).
[Round 14] Node 37 received an approval (2/2).
[Round 14] Node 37 starts phase 3. Sending token to left and right for 8 steps.
[Round 15] Node 29 sends token 37 forward (hop=7).
[Round 15] Node 21 sends token 37 forward (hop=7).
[Round 16] Node 12 sends token 37 forward (hop=6).
[Round 16] Node 25 sends token 37 forward (hop=6).
[Round 17] Node 12 sends token 37 forward (hop=5).
[Round 17] Node 25 sends token 37 forward (hop=5).
[Round 18] Node 29 sends token 37 forward (hop=4).
[Round 18] Node 21 sends token 37 forward (hop=4).
[Round 19] Node 37 received its token back. Became a leader.
[Round 19] Node 37 received its token back. Became a leader.
The end of the simulation. Winner is 37.
Winner: 37, total rounds: 19, number of messages: 53
```

Також емпіричне дослідження комунікаційної складності підтвердило теоретичні розрахунки. Кількість повідомлень зростає логарифмічно по відношенню до розміру кільця і стабільно тримається нижче теоретичної верхньої межі у $8n(\lceil \log_2 n \rceil + 1)$

Графік залежності кількості повідомлень від кількості вузлів:



4. Код програми

Код можна знайти за наступним посиланням (папка lab1):

`https://github.com/chesswondo/distributed-systems-labs`